Puteți propune probleme / rezolvări în modul "suggest" (apare cu verde). DN sau Radu Nichita dau ulterior approve dacă informația este corectă.

1. Cum arata graful pt problema cu fete si baieti, ce tip de graf e si cum se cuplajul

E un graf bipartit, care are o multime de noduri pentru 7, o multime de noduri pentru baieti, fetele in stanga, baietii dreapta. Exista muchie intre fata x si baiatul y daca 7. Y! JJj? , facem o retea de flux pe baza acestui graf bipartit, legand toate nodurile din stanga de o sursa, toate nodurile din dreapta de o destinatie, iar absolut toate.m. 76 h, hmmmuchiile din graf trebuie sa aiba capacitate 1

2. Cum ati folosi schema minimax pentru un joc de sah?

Aici ar merge eventual spus ca faci arborele starilor pana la o adancime anume, ca la un minimax obisnuit si ca dupa nu stiu cate mutari bagi o euristica pentru evaluarea starilor, idk. B 55

3. Enumerati utilizarile folosite in algoritmii de la curs ale timpilor de debut.. si de finish de la DFS

Apar pe la sortare topologica, la Kosaraju (CTC in general) si pe la componente biconexe (Tarjan).

4. Las Vegas vs Monte Carlo

Alea din lab

I a. Las Vegas are timp nedeterminat de rulare, Monte Carlo timp determinat (ex. il lasam sa ruleze X iteratii)

b. Las Vegas cand se opreste garanteaza solutie corecta/optima. La Monte Carlo cu cat il lasi sa ruleze mai mult cu atat solutia este mai aproape de cea corecta / cea optima.

5. ! I i. U.utilitatea algoritmului lui Johnson. De ce este mai bun decat alternativele.

Problema: drum de la n surse la m destinatii (in pricipiu la grafuri rare e mai bun comparativ cu Roy).

Solutii:

1. N x Dijkstra - O(M * N log N), dar cu costuri pozitive
2. g de la un nod fictiv - h[i] = distanta nod fictiv i
    a. La Johnson e ideea sa faci costurile pozitive ca sa poti aplica 1
    b. w[i][j] => w[i][j] + h[i] - h[j]
    c. Bagi 2 => d[i][j] = distanta calculata cu Dijkstra pe noul graf cu costuri pozitive
    d. d[i][j] => d[i][j] + h[j] - h[i] = distanta pe graful initial

13. Cum modifici Dijkstra daca graful este aciclic si orientat.

Cand graful este orientat si aciclic, putem sa sortam topologic graful si sa calculam acelasi vector de distante ca la Dijkstra parcurgand nodurile din sortarea topologica de la

stanga la dreapta si updatand d[vecin] = min(d[vecin], d[nod] + cost(nod, vecin))
d[sursa] = 0 initial, oo celelalte

14. Ce inseamna o euristica admisibila la A*? ca => 0 <= h(n) <= h*(n) oricare ar fi n
Ce s-ar intampla daca nu e admisibila? => A* ar gasi o solutie dar nu se garanteaza optimalitatea.

15. Evidentiati caracteristici DEI. Exemplificati pe x^n.

x ^ n = x ^ (n / 2) * x ^ (n / 2) daca n e par
x ^ n = x * x ^ (n - 1) daca n e impar
Complexitate: O(log n).
Caracteristicile DEI sunt mai sus, la comparatia cu DP

16. BF* vs A* (asta e foooarte prost explicata in curs, ma mai gandesc)

La BF*, g = 0, algoritm neoptim si complet, posibil sa ruleze la infinit pe grafuri infinite.
La A*, g = drumul de la sursa (ca la Dijkstra), algoritm complet si optim (depinde de euristica aleasa) .

17. Dati exemplu de un graf cu n noduri si m > n arce si o parcurgere in adancime care genereaza n arbori de parcurgere a grafului

Se poate desena o sortare topologica, iar dfs-ul se face de la dreapta la stanga. Se genereaza n arbori.

18. Cum se poate modifica numarul de CTCuri la adaugarea unei muchii? Justificati!

Numarul de CTCuri poate sa ramana la fel sau sa scada la adaugarea unei muchii, nu poate sa creasca. Numarul scade daca avem de exemplu un graf in forma de lant si muchia adaugata inchide ciclul (sau alte situatii similare), iar in celelalte situatii numarul ramane acelasi.

19. Transformand unul din algoritmii facuti la curs, scrieti in pseudocod un algoritm de detectare eficienta a existentei ciclurilor intr-un graf orientat. Justificati. Ce complexitate are?

Se poate face si cu DFS (se alege un nod random, se porneste DFS din el si se cauta cicluri asa, iar asta se face cat timp mai avem noduri nevizitate)
Data fiind exprimarea “transformand unul din algoritmii facuti la curs”, e posibil sa ceara o modificare a Bellman Ford, si anume: se pune cost -1 pe fie care muchie din graf si se porneste Bellman Ford, iar daca in urma rularii algoritmului gasim ca exista un ciclu de cost negativ, inseamna ca exista ciclu in graful nostru (automat orice ciclu din graf conduce la un ciclu de cost negativ)
Ca si complexitate, solutia cu DFS are O(N + M), iar cea cu Bellman Ford are O(N * M).

20. Cum se poate transforma o problema de determinare a fluxului maxim intr-o retea cu mai multe surse si mai multe destinatii intr-o problema de determinare a fluxului

maxim intr-o retea cu o sursa si o destinatie?

Se construieste o sursa noua, pe care o legam cu capacitate INF de toate sursele initiale,precum si o destinatie noua, pe care o legam cu capacitate INF de toate destinatiile initiale.

21. In ce cazuri complexitatea cautarii drumurilor de cost minim intr-un graf orientat G=(N,A) cu costuri pozitive este O(N + A)?

Cand graful este aciclic, deoarece avem cel mult un drum intre oricare 2 noduri si atunci costul e unic determinat.

22. Comparati A* si parcurgerea in latime.

Pe langa diferentele evidente, Trausanul s-a plans anul trecut ca nu a zis nimeni de Best F2 Best First, pe cand A* e un algoritm de cautare euristica.

23. Numarul maxim si minim de CTC-uri nebanale intr-un graf cu n noduri si a arce

Probs - minim 1

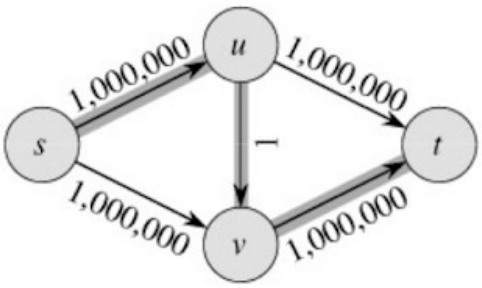
TODO - trebuie sa ma lamuresc ce ai voie sa faci si ce nu :)

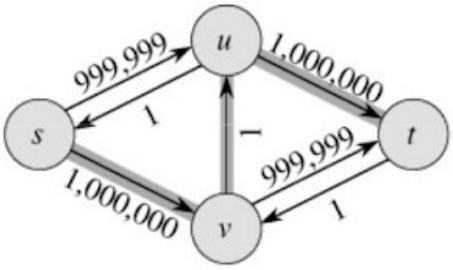
24. Pe ce caz merge Ford Fulkerson prost + exemplu

Chestia din curs cu 4 noduri si 5 muchii cu costurile de 1000 si 1.

Analiza

- Capacitati intregi – marire a |f| cu ≥ 1.
- Daca fluxul max e f*, atunci ≤ |f*| iteratii
  → O(a|f*|).
- Observati ca timpul de rulare este **nepolinomial** fata de marimea intrarii. Depinde de |f*|, care nu e o functie de |n| sau |a|.
- Daca capacitatile sunt rationale, se pot scala la intregi
- Daca sunt irationale, FORD-FULKERSON s-ar putea sa nu se termine!





Se alege drumul ala care contine 1 de fiecare data si se fac 1kk iteratii

25. ~~Greed~~y vs Divide et Impera = combinatie de DP vs DEI cu Greedy vs DP

Altele

1. **Ce este o cale reziduala si cum se poate obtine? Care este capacitatea reziduala a caii?**
   Exempificati.

Presupunem ca avem o retea de flux. Graful rezidual este un graf care contine doar muchiile nesaturate (muchie nesaturata = muchie cu capacitate != flux, adica muchie pe care se mai poate trimnuite flux). O cale reziduala este un drum care pleaca din sursa de la flux si merge pana in destinate (drena, whatever) doar pe muchii nesaturate (din graful rezidual).

Daca presupunem ca acea cale reziduala este v1 v2 … vk, unde v1 = sursa, vk = destinatia, atunci capacitatea reziduala a caii este min(capacitate[v[i + 1]][v[i]] - flux[v[i + 1]][v[i]]), i intre 1 si k - 1.

+ un exemplu random

2. **Ce tipuri de muchii se pot intalni intr-un graf neorientat?**

Fiind graf neorientat, se pot gasi intr-o parcurgere DFS doar 2 tipuri de muchii, **muchii de avans** (muchii de arbore), folosite pentru explorarea nodurilor; ele formează un graf parţial format din unul (graf conex) sau mai mulţi arbori ai parcurgerii **DF** si **muchii de intoarcere,** care unesc un nod cu un strămoş al său în arborele DF.

Pentru un graf orientat, sunt 4 tipuri de muchii:

• arce de arbore – care conduc la descoperirea de vârfuri noi
• arce de revenire – care leagă un vârf cu un strămoş în arborele DFS
• arce de înaintare – care leagă un vârf cu un descendent în arborele DFS
• arce de traversare – care leagă vârfuri din arbori DFS diferiţi (nu se încadrează în categoriile precedente).

3. **Care este numarul total de relaxari de muchii facute de catre Bellman Ford? Explicati din ce motiv algoritmul foloseste acest numar de relaxari de muchii (si nu mai multe sau mai putine)**

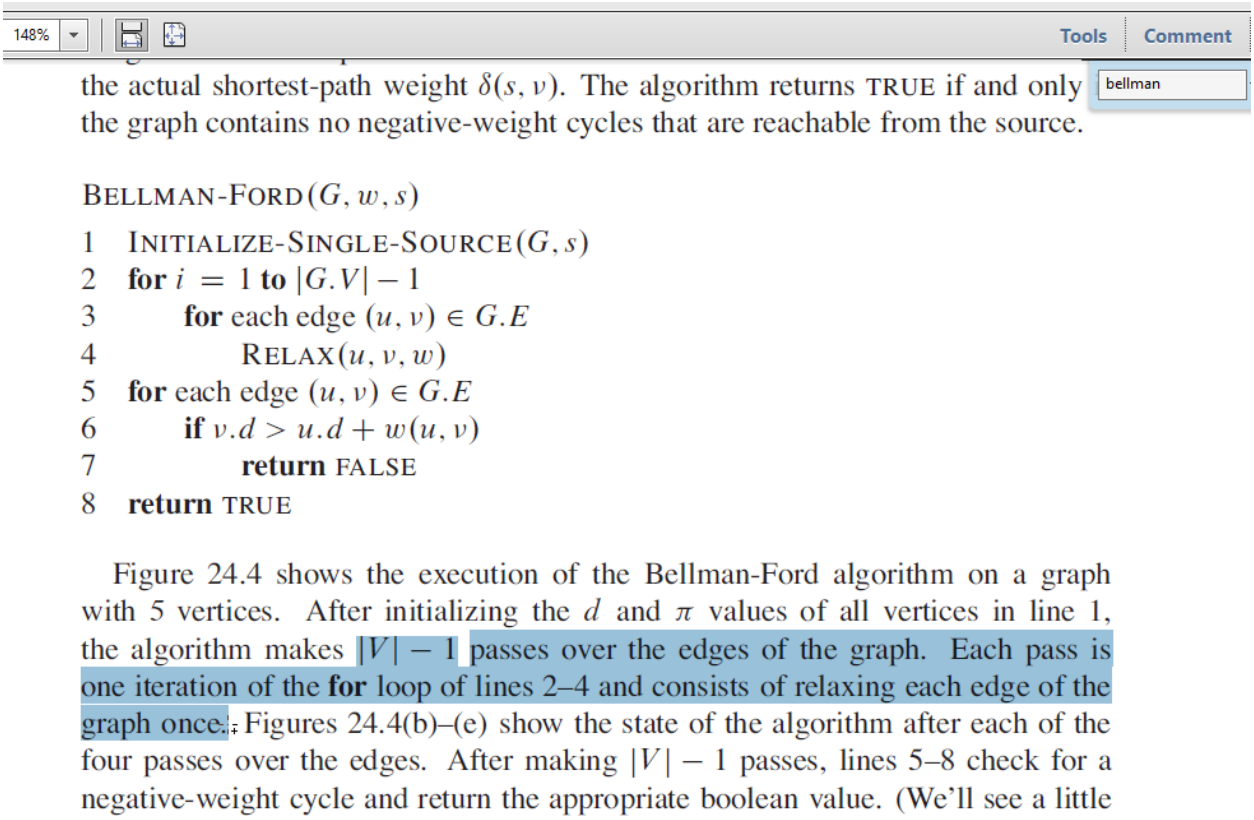
Numarul total de relaxari de muchii facute de Bellman Ford este (N - 2) * M (sau (N - 1) * M). Motivul pentru care algoritmul foloseste acest numar de relaxari de muchii este ca daca vrem sa detectam existenta unui ciclu negativ, atunci acest ciclu poate contine cel mult N noduri. Inainte sa facem Bellman Ford consideram cumva doar potentialele cicluri de lungime 2, apoi dupa k iteratii luam in calcul potentialele cicluri de lungime maxim k + 2, de aceea k trebuie sa fie N - 2. La final se mai face o iteratie prin care se incearca relaxarea muchiilor si in caz ca se gaseste una care inca mai updateaza o distanta, inseamna ca exista un ciclu negativ.

Motivul pentru care se face acest lucru este ca drumul minim dintre sursa si orice nod

destinație poate sa treacă prin maximum |V| noduri (adică toate nodurile grafului), respectiv |V|-1 muchii; prin urmare, relaxarea tuturor muchiilor de |V|-1 ori este suficienta pentru a propaga până la toate nodurile informația despre distanta minima de la sursa.

Daca, la sfârșitul acestor |E|*(|V|-1) relaxări, mai poate fi îmbunătățită o distanță, atunci graful are un ciclu de cost negativ si problema nu are soluție.

Cormen:



Deci ar trebui sa fie (V-1) * E ca sa afle distantele (dar asta ar trebui sa fie raspunsul uzual). Iar daca se cere cu tot cu verificarea ciclului V * E.

Adica cred ca sunt corecte amandoua abordarile, doar ca e vorba de o conventie: de ex pe wiki initial toate distantele sunt infinit, chiar si acolo unde are muchie. Si le numara inclusiv pe astea (cand foloseste muchia i->j sa spuna ca distanta de la i la j nu mai e infinit). Deci banuiesc ca daca le numeri si pe alea triviale sunt mai multe cu E…

P.S. pseudocodul din lab este acelasi cu cel de pe wiki, diferenta fiind fix asta (cu infinit pus degeaba in unele locuri).

4. **Cum se poate modifica alg lui Dijkstra daca graful este aciclic si orientat?**
   Cand graful este orientat si aciclic, putem sa sortam topologic graful si sa calculam

acelasi vector de distante ca la Dijkstra parcurgand nodurile din sortarea topologica de la stanga la dreapta si updatand d[vecin] = min(d[vecin], d[nod] + cost(nod, vecin))
d[sursa] = 0 initial, oo celelalte
al

5. **Daca facem +2 la costuri pe muchii / *2 la costuri, se modifica drumul minim gasit de Dijkstra?**

Daca facem *2 drumul ramane acelasi, doar i se dubleaza costul, iar daca facem +2 se modifica drumul. Spre exemplu:
1 -> 3 cu cost 4
1 -> 2 cu cost 2
2 -> 3 cu cost 1

Initial Dijkstra gaseste drumul 1 -> 2 -> 3 cu cost 3, dar dupa ce crestem costurile, costul drumului 1 -> 3 ajunge 6, pe cand 1 -> 2 -> 3 ajunge 7, ceea ce inseamna ca gasim alt drum.

6. **Ce se întâmplă cu algoritmul minimax cu tăieturi alfa-beta dacă modificăm condiția de tăiere din (β<α) în (β-k<α), unde k este o constantă oarecare. Discuție după k.**

In cazul in care:
β-k=β (k=0) => algoritmul nu se schimba;
β-k<β (k>0) => riscam sa facem taieturi inutile ce va duce la rezultate gresite in urma rularii algoritmului;
β-k>β (k<0) => riscam sa omitem taituri ce va duce la crestea timpului de rularea al algoritmului insa functionalitatea acestuia va ramane;

CA + CB (se repeta cateva de mai sus)
.
Raspuns
Raspuns dat partial
Nu sunt sigur de raspuns

Subiecte examen

1. Sa se spuna ce algoritm facut la curs poate sa determine eficient care este drumul de lungime maxima intr-un graf aciclic orientat cu lungimi pe arce si sa se schiteze idea folosirii acestuia si modificarile necesare pentru rezolvare.

Sortare topologica pentru a gasi sursa + dfs modificat din sursa care intoarce distanta maxima calculata pe parcurs

2. Sa se spuna ce schema de algoritm facuta la curs poate sa determine, cat mai eficient posibil, daca un careu de rebus are o asignare posibila de cuvinte (nu se considera definitiile cuvintelor). Sa se schiteze idea rezolvarii. Se presupune ca exista un dictionar in care cuvintele sunt reprezentate prin vectori de caractere. (Un careu de rebus este o matrice m x n cu unele celule goale). Toate celelalte celule trebuie sa primeasca cate o litera astfel incat secventele maximale de celule pe orizontala si verticala sa fie cuvinte valide).
Ceva cu constrangeri?

4. Explicati pe un exemplu simplu de ce algoritmul Dijkstra merge doar pentru costuri pozitive pe arce.(sigur nu e intrebarea gresita? Dijkstra merge pe costuri negative, atat timp cat nu avem cicluri de cost negativ - https://stackoverflow.com/a/25158544)

Recall that in Dijkstra's algorithm, **once a vertex is marked as "closed" (and out of the open set) - the algorithm found the shortest path to it**, and will never have to develop this node again - it assumes the path developed to this path is the shortest.

But with negative weights - it might not be true. For example:

`V={A,B,C} ; E = {(A,C,2), (A,B,5), (B,C,-10)}`

Dijkstra from A will first develop C, and will later fail to find `A->B->C`

`https://stackoverflow.com/questions/13159337/why-doesnt-dijkstras-algorithm-work-for-negative-weight-edge`



5. Fie un graf neorientat cu n noduri si a arce. Care este numarul minim si maxim de punti si puncte de articulatie ale acestui graf? Justificati. Dar daca graful are puncte de articulatie dar nu are punti?
I.
Nr maxim puncte de articulatie: n - 2, graful este un lant * - * - * - … - *, iar stergerea nodurilor

de la capete nu influenteaza numarul de componente conexe
Nr maxim punti:n - 1, graful este un lant * - * - * - … - *
Nr minim puncte de articulatie: 0, graful nu are muchii (sau 2-nod conex)
Nr minim punti: 0, graful nu are muchii (sau 2-arc conex)
II. graful are puncte de articulatie dar nu are punti
Nr maxim puncte de articulatie: n - 2, graful este un lant cu cate doua muchii intre un nod si vecinul lui * = * = * = … = *, iar stergerea nodurilor de la capete nu influenteaza numarul de componente conexe
Nr minim puncte de articulatie: 0, graful nu are muchii

11.Care este diferenta intre alg Las Vegas si Monte Carlo?las vegas

a. Las Vegas are timp nedeterminat de rulare, Monte Carlo timp determinat (ex. il lasi sa ruleze X iteratii)

b. Las Vegas cand se opreste garanteaza solutie corecta/optima. La Monte Carlo cu cat il lasi sa ruleze mai mult cu atat solutia este mai aproape de cea corecta / cea optima.

12.Schitati reteaua de restrictii pentru problema unei asignari de cifre pentru litere astfel incat sa fie valabila adunarea: SEND + MORE = MONEY

14.Sa consideram multimea elevilor de la un bal de absolvire. Pentru ultimul dans al balului, fiecare fata a facut o lista cu colegii cu care i-ar placea sa danseze. Spuneti cum ati putea construi un graf pentru aceasta problema, ce tip de graf este acesta si cu ce algoritm ati putea rezolva eficient problema de a gasi o asignare a perechilor de dans astfel incat sa fie un numar maxim de perechi. Precizam ca asignarea perechilor se face astfel incat orice fata sa nu danseze cu un coleg care nu e pe lista de preferinte.

E un graf bipartit, care are o multime de noduri pentru fete, o multime de noduri pentru baieti, fetele in stanga, baietii in dreapta. Exista muchie intre fata x si baiatul y daca fata x vrea sa danseze cu baiatul y.

Pentru a afla numarul maxim de perechi pe care le putem forma, facem o retea de flux pe baza acestui graf bipartit, legand toate nodurile din stanga de o sursa, toate nodurile din dreapta de o destinatie, iar absolut toate muchiile din graf trebuie sa aiba capacitate 1

16.Enumerati utilizarile folosite in algoritmii de la curs ale timpilor de debut si de finis de la parcurgerea in adancime a unui graf.
Apar pe la sortare topologica, la Kosaraju (CTC in general) si pe la componente biconexe (Tarjan).

17.Dati exemplu de un graf cu n noduri si m > n arce si o parcurgere in adancime care genereaza n arbori de parcurgere a grafului.
Fiecare nod e o componenta conexa si am mai multe drumuri de la fiecare nod la el insusi?
Sau 1->2->3->... (+ muchii de la 1 la 3, etc) si se face dfs pe noduri in ordine descrescatoare a

valorii.

19.Transformand unul din algoritmii facuti la curs, scrieti in pseudocod un algoritm de detectare eficienta a existentei ciclurilor intr-un graf orientat. Justificati. Ce complexitate are?

Se poate face si cu DFS (se alege un nod random, se porneste DFS din el si se cauta cicluri asa, iar asta se face cat timp mai avem noduri nevizitate)

Data fiind exprimarea “transformand unul din algoritmii facuti la curs”, e posibil sa ceara o modificare a Bellman Ford, si anume: se pune cost -1 pe fiecare muchie din graf si se porneste Bellman Ford, iar daca in urma rularii algoritmului gasim ca exista un ciclu de cost negativ, inseamna ca exista ciclu in graful nostru (automat orice ciclu din graf conduce la un ciclu de cost negativ)

Ca si complexitate, solutia cu DFS are O(N + M), iar cea cu Bellman Ford are O(N * M).

20.Cum se transforma o problema de determinare a fluxului maxim intr-o retea cu mai multe surse si mai multe destinatii intr-o problema de determinare a fluxului maxim intr-o retea cu o sursa si o destinatie.

Se construieste o sursa noua, pe care o legam cu capacitate INF de toate sursele initiale, precum si o destinatie noua, pe care o legam cu capacitate INF de toate destinatiile initiale.

21.Modificati un algortim de la curs astfel incat sa detecteze daca un graf e conex sau nu.
Nu am fost eu la curs, dar DFS/BFS sigur s-au facut. Faci BFS/DFS dintr-un nod oarecare. Daca la final toate nodurile sunt visited (colorate cu negru), atunci inseamna ca graful e conex. Daca nu, nu e conex.

22.Mentionati de ce e mai bun AC3 fata de AC1.
AC1 = O(n*r*a^3)
AC3 = O(r*a^3)
AC3 face mult mai puţine apeluri ale funcţiei REVISE decat AC1. Dacă o constrângere aduce modificări în reţea AC3 adauga pentru reverificare nodurile care punctează către nodul de plecare al restricţiei verificate, iar la AC1 se aplică algoritmul de consistenţa nodurilor şi apoi se aplică REVISE până nu se mai realizează nici o schimbare.

24.Care sunt pasii principali care trebuie urmati pentru a rezolva o problema folosind programarea dinamica? Exemplificati pe scurt aplicarea acestor pasi pe una dintre problemele discutate la curs./ Exemplificare pe problema parantezarii matricilor.

Algoritm generic

Soluții-parțiale^0 ← {Soluție-parțială-elementară$_1$, Soluție-parțială-elementară$_2$,...}

Pentru i=1 ***la*** n ***repetă***

Soluții-parțiale^i =combină(Soluții-parțiale$^{j<i}$,Criteriu-de-optim)

Întoarce Soluții-parțiale^n

Descompunere în subprobleme

- Încercăm să definim subprobleme identice cu problema originală, dar de dimensiune mai mică.

- $\forall$ $1 \le i \le j \le n$:
  - Notăm $A_{i,j} = A_i \times \ldots \times A_j$. $A_{i,j}$ are p_{i-1} linii si p_j coloane: $A_{i,j}(p_{i-1}, p_j)$
  - m[i, j] = numărul optim de înmulţiri pentru a rezolva subproblema $A_{i,j}$
  - s[i, j] = poziţia primei paranteze pentru subproblema $A_{i,j}$
  - Care e parantezarea optimă pentru $A_{i,j}$?

- Problema iniţială: $A_{1,n}$

Combinarea subproblemelor

- Pentru a rezolva $A_{i,j}$
  - trebuie găsit acel indice $i \le k < j$ care asigură parantezarea optimă:

$A_{i,j} = (A_i \times \ldots \times A_k) \times (A_{k+1} \times \ldots \times A_j)$

$A_{i,j} = A_{i,k} \times A_{k+1,j}$

Alegerea optimală

- Căutăm optimul dintre toate variantele posibile de alegere ($i \le k < j$)

- Pentru aceasta, trebuie însă ca şi subproblemele folosite să aibă soluţie optimală

(adică $A_{i,k}$ şi $A_{k+1,j}$)

26.Ce este o muchie inapoi (ce culori au varfurile ce determina muchia respectiva)? Demonstrati ca un graf aciclic nu are voie sa contina muchii inapoi in parcurgera in adancime.
O muchie de la un nod in curs de evaluare la un alt nod in curs de evaluare. Nu are voie sa - contina muchii inapoi pentru ca inseamna ca in cursul algoritmului dfs se gasesc muchii catre noduri care au fost deja evaluate, deci ca exista un ciclu. (Merge desen sau Metoda Reducerii la Absurd)

27.Schitati pe scurt pseudocodul pentru algorimtul minimax. Spuneti apoi minim 2 posibiliati prin care acesta poate fi imbunatatit pentru a reduce timpul de rezolvare.

```
01  function minimax(node, depth, maximizingPlayer)
02      if depth = 0 or node is a terminal node
03          return the heuristic value of node

04      if maximizingPlayer
05          bestValue := −∞
06          for each child of node
07              v := minimax(child, depth − 1, FALSE)
08              bestValue := max(bestValue, v)
09          return bestValue

10      else                (* minimizing player *)
11          bestValue := +∞
12          for each child of node
13              v := minimax(child, depth − 1, TRUE)
14              bestValue := min(bestValue, v)
15      return bestValue
```

Doua posibilitati pentru a reduce timpul de rezolvare:
Conteaza foarte mult ordinea in care analizam mutarile posibile => Sortarea mutarilor ce vor fi analizate
Limitarea adancirii cautarii (atat in mod dinamic cat si in mod static)

28.In ce cazuri complexitatea cautarii drumurilor de cost minim intr-un graf orientat G=(N, A) cu costuri pozitive este O(#N + #A)?

https://www.geeksforgeeks.org/shortest-path-for-directed-acyclic-graphs/

Pentru un graf aciclic orientat (DAG) se poate face o sortare topologica si apoi distantele se pot calcula folosind DFS. Se ajunge la o complexitate de O(V + E).

Cand toate costurile dintr-un graf oarecare sunt egale (in acest caz putem aplica BFS ca sa aflam costul minim).

29.Ce este o cale reziduala intr-o retea de conducte si cum se poate obtine? Care este

capacitatea reziduala a caii si cum se poate determina? Exemplificati.

30.Comparati A* si cautarea in latime.
Cautarea in latime pur si simplu se uita pas cu pas, mai departe la care e urmatorul “nod” din graf si verifica daca a ajuns unde trebuie. La a* se foloseste o euristica care daca este aleasa corect duce mult mai rapid la rezolvarea problemei. Merge un desen aici in care sa arati ca la BFS se cauta in jurul punctului de start (Faci un punct cu cercuri in jurul lui), iar la un A* ok se duce efectiv in directia generala a destinatiei.

31.Ce tip de algoritmi ati folosi daca ar trebui sa calculati care este unul dintre cele mai mari numere prime?
Abordare de tipul Las Vegas. (poate MonteCarlo, cf curs?)

Mica teorema a lui Fermat: Daca n este prim -> oricare ar fi X, 0 < X < n, X^(n - 1)%n=1.
Luam aleator un numar de la 0 la n si verificam relatia lui Fermat. Daca da false, numarul nu este prim. Continuam sa luam aleator cate-un numar pana cand demosntram ca numarul nu este prim, sau pana epuizam toate numerele, adica am descoperit un numar prim.
Sau Miller-Rabin, dar e mizerie ala.

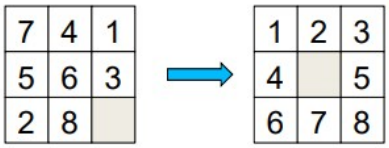
32.Evidentiati etapele cele mai importante ale strategiei de rezolvare divide et impera, dupa care explicati cum poate fi folosita aceasta pentru a calcula eficient x pow n (x ^ n), pentru x – numar real, iar n >= 0.
E explicat mai sus

33.Explicati utilitatea algoritmilor de satisfacere/propagare a constrangerilor, pornind de la functionalitatea oferita de REVISE. Trecand apoi in revista algoritmul AC1, explicati de ce acesta este considerat ineficient.
Functia revise sterge acele valori din domeniul de definiţie al unei variabile pentru care nu este satisfăcută restricţia pentru nici o valoare corespunzătoare celeilalte variabile a restricţiei. Astfel acele ramuri nu mai sunt parcurse.
AC1 este ineficient deoarece se aplică REVISE până nu se mai realizează nici o schimbare, nu se face reverificarea nodurilor direct implicate de o constrângere din reţea.

34.Definiti notiunea de dominanta pentru algoritmul A*, explicand de ce aceasta este importanta in alegerea euristicii. Dati exemplu de o problema unde puteti defini (cel putin) doua euristici diferite si discutati care dintre ele este dominanta.
Un algoritm A1 * domină un algoritm A2 * dacă orice nod expandat de A1 * este expandat si de A2 *. (eventual, A2 * expandează noduri suplimentare față de A1 *, deci A1 * poate fi mai rapid DECAT A2 *.)

Dominanța - Exemplu

- Considerăm jocul 8-pătrățele care trebuie aranjat pornind de la forma inițială prin mutarea locului 'liber' astfel incat să ajungem la forma finală:



- Două euristici posibile:
  - h_1 = numărul pătrățelelor a căror poziție curentă diferă de poziția finală;
  - $h_1 = \Sigma_{p \in piese}(\delta_p)$, unde $\delta_p = 0$ dacă poziția curenta coincide cu cea finala si $\delta_p = 1$, altfel
  - h_2 = distanța Manhattan = suma distanțelor pe verticală si orizontală intre pozițiile curente ale pătrățelelor si pozițiile lor finale
  - $h_2 = \Sigma_{p \in piese}(dist_h_p + dist_v_p)$

h2 e mai informata decat h1

35.Dati exemplu de un algoritm studiat la curs care poate fi utilizat pentru determinarea ciclurilor de cost negativ intr-un graf orientat. Dupa aceea, scrieti pseudocodul pentru determinarea tuturor nodurilor care alcatuiesc un astfel de ciclu SAU demonstrati corectitudinea determinarii ciclului negativ de catre acest algoritm.
Floyd Warshall si daca pe diagonala principala apar valori negative inseamna ca avem ciclu de cost negativ, cred?

37.Care este inaltimea maxima la care poate ajunge un nod in cadrul algoritmilor de tip preflux (push-relabel) folositi pentru determinarea fluxului maxim. Argumentati pe scurt./ + De ce este utilă/necesară această limită?
Inaltimea maxima este 2 * |V| - 1 deoarece aceasta este lungimea maxima posibila a unui drum rezidual.

38.Dandu-se matricea finala de distante obtinunta folosind algoritimul Floyd-Warshall, se poate stabili daca graful respectiv are un ciclu de cost negativ? Argumentati.

In mod normal, matricea ar trebui sa aiba 0 pe diagonala principala. In cazul unui ciclu negativ, pe diagonala pot aparea valori diferite de 0 deoarece daca plec dintr-un nod si urmez ciclul, la final voi ajunge tot in el, si voi avea un cost < 0, fiind ciclu negativ, deci imi va pune noua valoare in matricea finala (ceva negativ < 0).

39.Demonstrati ca un graf are cicluri daca si numai daca in timpul unei parcurgeri DFS sunt

descoperite arce inapoi.
Dfs => cycle : in timpul parcurgerii dfs se gaseste o muchie noua catre un nod “gri” => exista o muchie care duce care a fost deja marcat ca fiind in curs de vizitare => exista ciclu (merge desen aici)
Cycle => dfs : exista ciclu => exista macar o muchie care se “intoarce” la un nod care va fi marcat ca “gri” deci care a fost marcat ca in curs de vizitare => parcurgerea dfs va gasi ac Vea muchie. (merge desen, dar ar fi cam acelasi lucru)

40.Explicati utilitatea taieturilor alfa-beta pentru arborii de joc si dati un exemplu de arbore de joc in care sa evidentiati cel putin 2 taieturi:una pe nivelul MIN, a doua pe nivelul MAX./ Explicați ce reprezintă o tăietura alfa-beta într-un arbore de joc și dați un exemplu de astfel de arbore, cu maxim 3 nivele, în care să scoateți în evidență minim două tăieturi alfa-beta.

44.Pornind de la definiția funcției REVISE, arătați de ce algoritmul AC3 este mai perfomant decât algoritmul AC1.
AC1 = O(n*r*a3)
AC3 = O(r*a3)
AC3 face mult mai puţine apeluri ale funcţiei REVISE decat AC1. Dacă o constrângere aduce modificări în reţea AC3 adauga pentru reverificare nodurile care punctează către nodul de plecare al restricţiei verificate, iar la AC1 se aplică algoritmul de consistenţa nodurilor şi apoi se aplică REVISE până nu se mai realizează nici o schimbare.

46.Ce se întâmplă cu algoritmul minimax cu tăieturi alfa-beta dacă modificăm condiția de tăiere din (β<α) în (β-k<α), unde k este o constantă oarecare. Discuție după k.

In cazul in care:

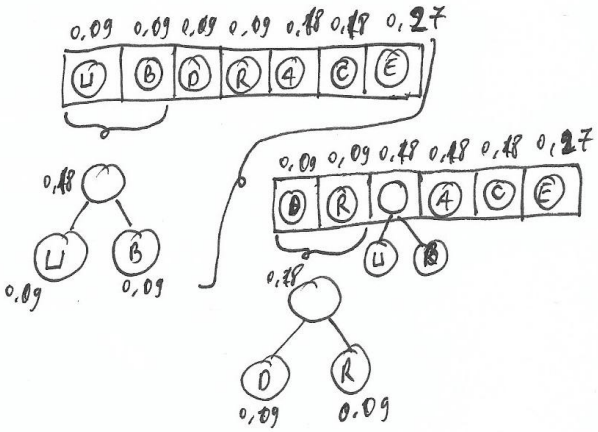
1. k=0 => algoritmul nu se schimba;
2. β-k<β (k>0) => riscam sa facem taieturi inutile ce va duce la rezultate gresite in urma rularii algoritmului;
3. β-k>β (k<0) => riscam sa omitem taituri ce va duce la crestea timpului de rularea al algoritmului insa functionalitatea acestuia va ramane;

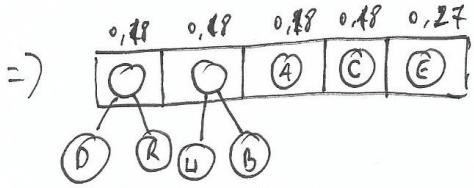
47.Construiti arborele Huffman de compresie pentru urmatorul text “ce abecedar” si calculati care este costul acestuia (cu 2 zecimale).

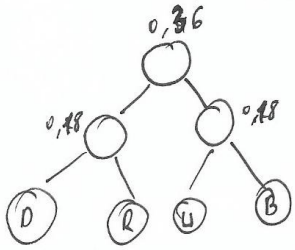
+

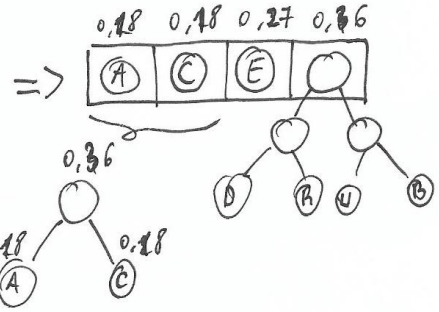
CE␣ABECEDAR

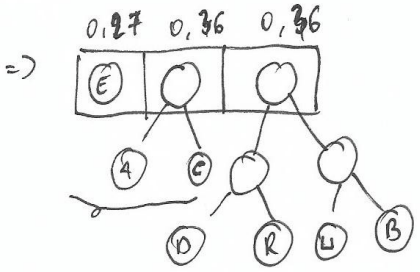
E – 3/11 = 0,27
C – 2/11 = 0,18
A – 2/11 = 0,18
␣ – 1/11 = 0,09
B – 1/11 = 0,09
D – 1/11 = 0,09
R – 1/11 = 0,09

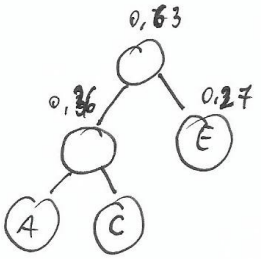


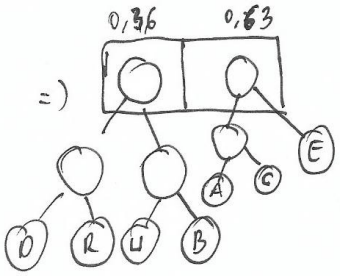


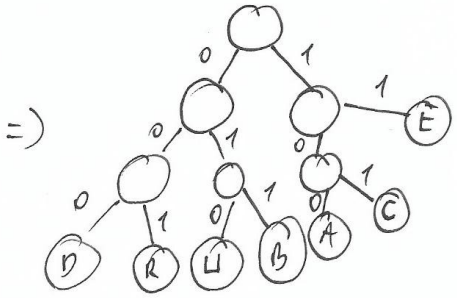










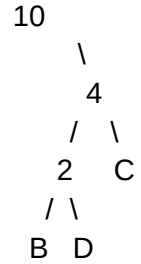


aSortam literele dupa numarul de aparitii:
E - 3
C - 2
A - 2
Spatiu - 1
B - 1
D - 1
R - 1

A aparut si R, acum cum faci arborele? Ca initial l-ati uitat.

Initial fiecare litera face parte dintr-un subarbore cu un nod.
Cat timp avem cel putin 2 subarbori, luam subarborii cu cele mai putin aparitii si ii unim intr-un arbore cu numar de aparitii egal cu suma nr celor doi subarbori

Pentru exemplul dat, in final arborele Huffman va arata asa:



Importante sunt doar literele si caracterul spatiu.
Pentru a determina incodarea huffman pentru fiecare caracter, procedam in felul urmator: pornim din radacina, si continuam in jos pentru a ajunge la caracterul respectiv. Daca am luat-o prin fiul stang al nodului curent, adaugam un 0 la encodare; altfel, adaugam un 1

Ex: vrem sa encodam pe A. pentru a ajunge la el va trebuie sa mergem: stanga -> stanga -> dreapta, deci incodarea Huffman este 001

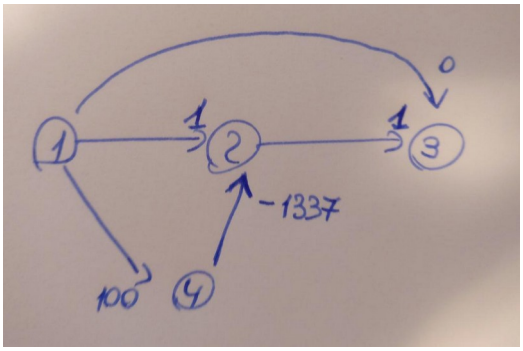
Pentru restul caracterelor, incodarea Huffman este:
E - 01
C - 11
‘ ‘ - 000
B - 100
D - 101

Se observa ca aceasta incodare asigneaza coduri mai scurte caracterelor care apar mai des in mesaj si, in plus, niciun cod nu este prefixul altuia.

Legat de cost, aici (http://dept.cs.williams.edu/~tom/weavingCS/s07/homeworks/Hw7.pdf) zice

ca efectiv faci traducerea din caractere in coduri huffman si vezi care e lungimea. Dar in cazul asta nu stiu de ce zice Trausan ca vrea cu 2 zecimale avand in vedere ca facem doar adunari :| <- trausan vrea sa faci arborele nu dupa nr de aparitii ci dupa probabilitatea aparitiei, adica A - 2/10, B - 1/10 etc si costul arborelui e suma de probabilitate litera * nr de biti (3/10 * 2 pt E)

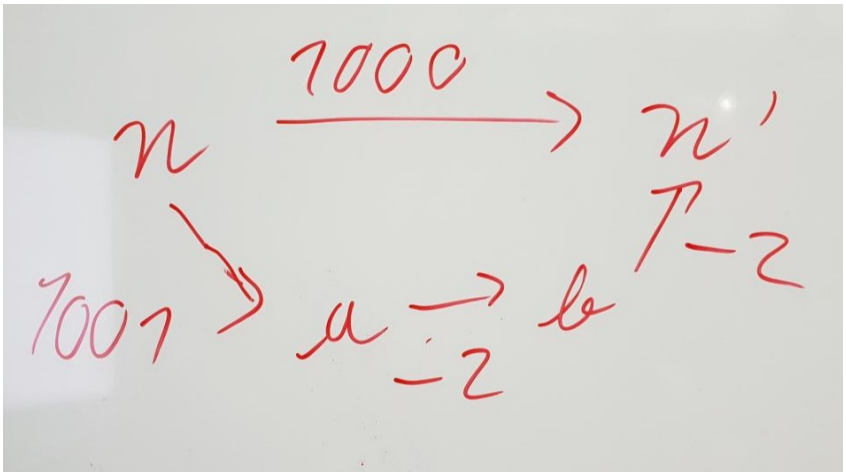
48.Dati un exemplu de graf cu maxim 5 noduri si cel putin o muchie de cost negativ pentru care algoritmul lui Dijkstra pentru drumuri minime de sursa unica calculeaza gresit costul drumurilor minime (este suficient sa scrieti pentru un nod costul corect si cel calculat de algoritm). Cum se poate modifica algoritmul pentru a rezolva aceasta problema?



Solutie: verificam si pentru nodurile din closed (facem Bellman Ford)

49.Taieturile alfa-beta reduc complexitatea unei rezolvari cu minimax pentru jocuri doar daca sunt folosite impreuna cu o alta tehnica. Care este aceasta si de ce este importanta? Dati un scurt exemplu care demonstreaza ca folosirea acesteia impreuna cu alfa-beta aduce imbunatatiri.
?Nu mai generam anumite ramuri cand stim ca min/max oricum nu o vor lua pe acolo. Astfel se reduce timpul de calcul.
?Folosirea unei euristici care sa parcurga subarbori care garanteaza o taiere alfa-beta, astfel restul subarborilor din la acel nivel nu mai trebuie parcursi.
?Limitarea adâncimii căutării = căutare în adâncime mărind iterativ adâncimea maximă până la care căutăm

50.Daca pentru fiecare nod n din cadrul unei explorari folosind algoritmul A* euristica este definita in felul urmator: h(n) = max{cost(n, n') | n' $\in$ succs(n)} (costul muchiei de cost maxim care pleaca din n catre un succesor al sau). Ce puteti spune despre aceasta euristica si despre rezultatul intors de catre algoritm?
h nu e admisibila, ex:



h*(n) = 1001 - 2 - 2 = 997
h(n) = max{1000, 1001} = 1001
Deci nu este garantat ca algoritmul va intoarce o solutie optima.

52.Dați exemplu de o problemă care se poate rezolva, eventual cu constrângeri diferite asupra datelor de intrare, atât folosind programare dinamică, cât și cu programare lacomă. Explicați diferențele și asemănările între cei doi algoritmi (inclusiv complexitatea lor).
Problema rucsacului

- Greedy: se pot lua doar bucati dintr-un anumit obiect
  alege in ordine descrescatoare in functie de pret/greutate unitati din obiecte
  T(nlogn), S(1)
- DP: obiectele nu se pot sectiona
  Construieste dp[i][j] unde dp[i][j] reprezinta valoarea maxima care se poate obtine umpland un ghiozdan de capacitate j folosind doar primele i obiecte
  T(n * capacitate), S(n * capacitate)

54.Fie un algoritm A* ghidat de următoarea euristică, h(n) = 0 ∀ n ∈ V, pentru problema 8-puzzle discutată la curs. Explicați care este numărul maxim de noduri care sunt scoase din mulțimea CLOSED pentru a fi reintroduse în mulțimea OPEN în timpul găsirii unei secvențe de pași de la o configurație inițială a problemei la o configurație finală. Asigură această euristică găsirea soluției optime ca număr de pași efectuați?
Euristica este admisibila, deci A*gaseste solutia optima ca numar de pasi efectuati.

55.Explicati utilitatea algoritmului lui Johnson. De ce este mai bun decat alternativele.
Problema: drum de la n surse la m destinatii.
Solutii:

1. RoyFloyd - O(n^3)
2. Nx Dijkstra - O(N * M log N), dar cu costuri pozitive
3. Johnson e smenul sa faci costurile pozitive ca sa poti aplica 2

e. BellmanFord de la un nod fictiv - h[i] = distanta nod fictiv i
f. w[i][j] => w[i][j] + h[i] - h[j]
g. Bagi 2 => d[i][j] = distanta calculata cu Dijkstra pe noul graf cu costuri pozitive
h. d[i][j] => d[i][j] + h[j] - h[i] = distanta pe graful initial

57.Daca facem +2 la costuri pe muchii / *2 la costuri, se modifica drumul minim gasit de Dijkstra?

Daca facem *2 drumul ramane acelasi, doar i se dubleaza costul, iar daca facem +2 se modifica drumul. Spre exemplu:
1 -> 3 cu cost 4
1 -> 2 cu cost 2
2 -> 3 cu cost 1

Initial Dijkstra gaseste drumul 1 -> 2 -> 3 cu cost 3, dar dupa ce crestem costurile, costul drumului 1 -> 3 ajunge 6, pe cand 1 -> 2 -> 3 ajunge 7, ceea ce inseamna ca gasim alt drum.